

Отчёт



Первым делом я модернизировала код и добавила три независимых режима запуска: PROFILE — профилирование по времени и памяти; TRAIN — обучение и измерение пропускной способности; VALIDATE — вычисление итоговой метрики MAE на валидационной выборке.

Профилирование проходит так: пропуск первые 5 итераций, далее 1 итерация для прогрева, затем запись следующих 10, фиксируя CPU/CUDA-время, выделение памяти, формы тензоров и тд.

Посмотрим на итоги рофилирования до начала оптимизации:

Основные операции по CUDA-time

Name	Self CUDA	Self CUDA %	Calls	Input Shapes
forward	3.921 s	62.78%	10	[]
cell_weighted_sum	3.201 s	51.25%	60	[]
aten::mul	3.086 s	49.40%	60	[8, 128, 984, 1], [8, 128, 984, 128]
cell_expansion	669.596 ms	10.72%	60	[]
aten::copy_	580.115 ms	9.29%	60	[8, 984, 128, 128]
aten::addmm	412.275 ms	6.60%	60	[1007616, 1], [1, 128]

Основные операции по CUDA-memory

Name	CUDA Mem	Calls	Input Shapes
aten::empty	30.33 GB	1930	[], [], [], [], [], []
aten::addmm	28.83 GB	60	[128], [1007616, 1], [1, 128]
aten::add	28.83 GB	60	[8, 1, 984, 128], [8, 128, 984, 128]
aten::mul	28.83 GB	60	[8, 128, 984, 1], [8, 128, 984, 128]
aten::mul	28.83 GB	60	[8, 128, 984, 128], [8, 128, 984, 1]

Видно, что узкое место находится в `cell_expansion` и `cell_weighted_sum` (`def forward` в `class TromptCell(nn.Module)`): внутри них создаётся и несколько раз обрабатывается тензор размера $[B, P, C, D]$ ($B = 8$ — размер батча, $C = 984$ — количество столбцов, $P = 128$ — количество промптов, $D = 128$ — размерность эмбединга). Попробуем упростить математически и найти эквивалентную запись.

Введём обозначения для матриц (чтобы согласовать с кодом):

$$\begin{aligned} X &= x, K = x_{\text{emb}}, \\ Y &= x_{\text{prompt}}, Z = x_{\text{column}}, \\ M &= \text{mask}, F = x_{\text{out}} \\ X &\in \mathbb{R}^{B \times C}, K \in \mathbb{R}^{B \times C \times D}, Y \in \mathbb{R}^{B \times P \times D}, \\ Z &\in \mathbb{R}^{B \times C \times D}, M \in \mathbb{R}^{B \times P \times C}, F \in \mathbb{R}^{B \times P \times D} \end{aligned}$$

Разберем код по строкам:

```
x_prompt @ x_column.transpose(1, 2)
```

$$S_{bpc} = \sum_{d=1}^D Y_{bpd} Z_{bcd}, \quad S \in \mathbb{R}^{B \times P \times C}$$

```
mask = torch.softmax(S, dim=-1)
```

$$M_{bpc} = \frac{\exp(S_{bpc})}{\sum_{j=1}^C \exp(S_{bpj})}$$

```
x_emb = x_emb.unsqueeze(1) + (self.dense_expand(x_emb.unsqueeze(-1)).permute(0, 3, 1, 2))
```

Пусть параметры слоя `self.dense_expand = nn.Linear(1, n_prompts)` равны $W \in \mathbb{R}^{P \times 1}$ и $q \in \mathbb{R}^P$, а $a_p = W_{p,1}$. Тогда после линейного слоя для промпта p получаем $a_p K_{bcd} + q_p$, то есть тензор $\mathbb{R}^{B \times C \times D \times P}$. Операция `permute(0, 3, 1, 2)` меняет только порядок осей: $(B, C, D, P) \rightarrow (B, P, C, D)$

$$(B, 1, C, D) + (B, P, C, D) \rightarrow (B, P, C, D)$$

Пусть полученный тензор через T . Тогда

$$\begin{aligned} T_{bpcd} &= K_{bcd} + (a_p K_{bcd} + q_p) \\ &= K_{bcd} + a_p K_{bcd} + q_p \\ &= (1 + a_p) K_{bcd} + q_p, \quad T \in \mathbb{R}^{B \times P \times C \times D} \end{aligned}$$

```
x_out = (mask.unsqueeze(-1) * x_emb).sum(dim=2)
```

$$\begin{aligned} F_{bpd} &= \sum_{c=1}^C M_{bpc} T_{bpcd}, \quad F \in \mathbb{R}^{B \times P \times D} \\ F_{bpd} &= \sum_{c=1}^C M_{bpc} ((1 + a_p) K_{bcd} + q_p) \\ &= \sum_{c=1}^C M_{bpc} (1 + a_p) K_{bcd} + \sum_{c=1}^C M_{bpc} q_p \\ &= (1 + a_p) \sum_{c=1}^C M_{bpc} K_{bcd} + q_p \sum_{c=1}^C M_{bpc} \end{aligned}$$

Так как softmax выполняется по C , то для каждого b, p : $\sum_{c=1}^C M_{bpc} = 1$, поэтому упрощаем до такого:

$$F_{bpd} = (1 + a_p) \sum_{c=1}^C M_{bpc} K_{bcd} + q_p$$

Получается, что избавились от создания огромного тензора: $T \in \mathbb{R}^{B \times P \times C \times D}$

```
class TromptCell(nn.Module):
    ...

    def forward(self, x: torch.Tensor, prev_cell_out: torch.Tensor) -> torch.Tensor:
        ...
        a = self.dense_expand.weight[:, 0].view(1, -1, 1)
        q = self.dense_expand.bias.view(1, -1, 1)

        x_out = (1.0 + a) * (mask @ x_emb) + q
        return x_out
```

После замены я повторила профилирование и вот что вышло:

Основные операции по CUDA-time

Name	Self CUDA	Self %	CUDA total	CUDA avg	Calls
forward	162.493 ms	87.78%	162.493 ms	16.249 ms	10
Optimizer.step#AdamW.step	37.139 ms	20.06%	37.139 ms	3.714 ms	10
aten::native_layer_norm_backward	27.497 ms	14.85%	32.698 ms	272.482 μ s	120
cell_importance_getter	0 μ s	0.00%	28.777 ms	479.623 μ s	60
aten::bmm [B,P,C]@[B,C,D]	28.468 ms	15.38%	28.468 ms	237.234 μ s	120
aten::bmm [B,P,D]@[B,P,C]	24.528 ms	13.25%	24.528 ms	136.268 μ s	180
cell_feature_embedding	21.677 ms	11.71%	21.677 ms	361.286 μ s	60
cell_weighted_sum	20.925 ms	11.30%	20.925 ms	348.746 μ s	60
column_normalization	19.518 ms	10.54%	19.518 ms	325.295 μ s	60
aten::native_layer_norm	19.468 ms	10.52%	19.468 ms	162.233 μ s	120

Основные операции по CUDA-memory

Name	CUDA Mem	Self CUDA Mem	Calls
aten::empty	1.52 GB	1.52 GB	1910
aten::bmm	730.94 MB	730.94 MB	180
aten::_softmax	269.69 MB	269.69 MB	60
aten::mul	234.06 MB	234.06 MB	60
aten::add	230.62 MB	230.62 MB	60

aten::clamp_min	230.62 MB	230.62 MB	60
aten::bmm	230.62 MB	230.62 MB	60
aten::mul	230.62 MB	230.62 MB	60
aten::threshold_backward	230.62 MB	230.62 MB	60
aten::mul	230.62 MB	230.62 MB	60
aten::empty_strided	99.59 MB	99.59 MB	960

Заметим, что forward за 10 итераций сократился с 3.921 s до 162.493 ms. Время cell_weighted_sum за 60 вызовов сократилось с 3.201 s до 20.925 ms. И теперь узким местом является cell_importance_getter (в коде это опять def forward в class TromptCell(nn.Module)). По размерностям тензора я поняла, что надо оптимизировать эту строчку:

```
x_column = self.ln_col(
    self.emb_column.unsqueeze(0).repeat(x_emb.shape[0], 1, 1)
)
```

Что происходит: self.emb_column не зависит от объектов батча, операция repeat просто создаёт В одинаковых копий одной и той же матрицы, после этого к каждой копии независимо применяется один и тот же LayerNorm, а так как все копии одинаковые, результаты нормализации также будут одинаковыми, поэтому нет необходимости сначала копировать self.emb_column по batch, а потом В раз выполнять одинаковую нормализацию. Хочу так:

```
x_column = self.ln_col(self.emb_column)
```

Захотела такую же штуку проверить и тут:

```
x_prompt = self.emb_prompt.unsqueeze(0).repeat(x_emb.shape[0], 1, 1)
```

и тут

```
x_prompt = self.prompt.unsqueeze(0).repeat(x.shape[0], 1, 1)
```

И self.emb_prompt, и self.prompt являются параметрами модели и не зависят от конкретного объекта батча, поэтому repeat снова создаёт несколько одинаковых копий. Потом над ними делается — LayerNorm, конкатенация и dense_imp — применяются одинаково к каждой копии, следовательно, результат для каждого элемента батча также получается одинаковым. Вот такие изменения получились:

```
# Trompt.forward
x_prompt = self.prompt

# TromptCell.forward
x_prompt = self.emb_prompt

x_prompt = self.dense_imp(
    torch.cat([self.ln_prompt(x_prompt), prev_cell_out], dim=-1)
) + x_prompt

x_column = self.ln_col(self.emb_column)

scores = x_prompt @ x_column.T
mask = torch.softmax(scores, dim=-1)
```

Основные операции по CUDA time

Name	Self CUDA	Self %	CUDA avg	Calls
forward	123.859 ms	111.99%	12.386 ms	10
Optimizer.step#AdamW.step	39.352 ms	35.58%	3.935 ms	10
cell_feature_embedding	24.706 ms	22.34%	411.766 µs	60
cell_weighted_sum	21.157 ms	19.13%	352.623 µs	60
importance_dense	13.397 ms	12.11%	223.288 µs	60
aten::native_layer_norm_backward	12.349 ms	11.17%	281.286 µs	60

Основные суммарные затраты CUDA-памяти

Name	CUDA Mem	Self CUDA Mem	Calls
aten::empty	1.06 GB	1.06 GB	1830

aten::mul	237.50 MB	237.50 MB	60
aten::add	234.06 MB	234.06 MB	60
aten::clamp_min	234.06 MB	234.06 MB	60
aten::mm	234.06 MB	234.06 MB	60
aten::threshold_backward	234.06 MB	234.06 MB	60
aten::_softmax	28.83 MB	28.83 MB	60

После оптимизации время forward за 10 итераций уменьшилось с 162.493 ms до 123.859 ms. И теперь основное время приходится на cell_feature_embedding, cell_weighted_sum, importance_dense. Перед следующей оптимизацией я очень захотела проверить скорость полного обучения, тут я на всякий случай (сугубо для оптимизации) увеличила батч до 128. Вышло, что одна эпоха заняла 94.43 s. Скорость я считала как число обработанных за эпоху объектов, делённое на время эпохи. Перед запуском таймера и после последнего батча вызывала torch.cuda.synchronize(), чтобы измерить и работу GPU:

```
Epoch 1/1; loss = 0.13398; time = 94.43s;
throughput = 1694.60 samples/sec
```

Таким образом, скорость обучения составила примерно 1695 samples/sec. Мой бейзлайн перед всеми оптимизациями работал ~ 14 samples/sec (13.27 samples/sec). Следовательно, относительно исходной реализации получено ускорение примерно в ≈ 121 раз. Таким образом сейчас уже побит первый чекпоинт.

Далее я решила попробовать torch.compile. и использовала режим

```
model.compile(mode="reduce-overhead")
```

Первые вызовы с torch.compile шли существенно медленнее, поскольку в этот момент происходит построение и компиляция графа, поэтому перед измерением скорости я выполнила 5 прогревочных forward и backward, не выполняя optimizer.step(). Получилось, что без torch.compile обучение одной эпохи занимало 94.43 s при throughput 1694.60 samples/sec, а вот с ним время уменьшилось до 85.91 s, а throughput вырос до 1862.70 samples/sec

Затем я решила добавить AMP, вот так вот сделала:

```
with torch.autocast(device_type="cuda", dtype=torch.float16, enabled=AMP):
    pred = model(x)
    loss = F.mse_loss(pred, target)
```

и еще юзнула это, чтобы не было потери слишком маленьких значений градиентов:

```
scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()
```

Время обучения составило 33.06 s, а throughput вырос до 4840.59 samples/sec. Помним, что до добавления AMP throughput составлял 1862.70 samples/sec, поэтому AMP дал дополнительное ускорение примерно в ≈ 2.60 раза, а относительно исходной скорости около 14 samples/sec суммарное ускорение на данном этапе составляет примерно ≈ 346 раз.

Далее я распределила обучение на две Tesla T4 с помощью DistributedDataParallel: на каждой GPU использовался батч по 64 объекта. Для двух GPU я суммировала число обработанных объектов на обоих процессах и делила его на время эпохи, измеренное после синхронизации процессов, пиковая скорость — максимум среди эпох. У нас первая эпоха заняла 75.65 s (2115.32 samples/sec), а следующие четыре — от 22.41 до 23.09 s каждая. Пиковая скорость достигла 7140.68 samples/sec, средняя за все пять эпох с учётом медленной первой — 6058.58 samples/sec, а средняя по эпохам 2–5 — 7044.40 samples/sec. Значит, относительно исходной реализации (13.27 samples/sec) — примерно 538 раз стало быстрее. На валидации прочекала, что итоговая MAE = 0.12358.

Затем я увеличила батч с 64 до 128 объектов на каждую GPU (общий батч 256). Получила, что первая эпоха заняла 72.97 s, а следующие четыре — от 17.57 до 17.65 s. Пиковая скорость выросла до 9109.43 samples/sec, средняя за пять эпох составила 7710.57 samples/sec. Это в ≈ 1.28 раза быстрее пика с батчем 64 и примерно в 686 раз быстрее исходных 13.27 samples/sec. MAE на валидации составила 0.12434